

PRATİK TARİFLER — TEKNİK KILAVUZ

Görsel Üretim ve Standardizasyon Kılavuzu

Tarif & Malzeme Görsellerinin Üretimi

API Kanalları · AI Üretim · Standardizasyon Kuralları · Script Kodu

İçindekiler

1 • Genel Bakış ve Strateji	3
1.1 Hibrit Kaynak Yaklaşımı	3
1.2 Maliyet ve Dağılım Tahmini	3
2 • Görsel Kaynak Kanalları ve API Anahtarları	4
2.1 Unsplash API	4
2.2 Pexels API	5
2.3 Pixabay API	5
2.4 OpenAI DALL·E 3 (AI Üretim)	6
2.5 Flux Pro — Alternatif AI	7
2.6 API Anahtarı Yapılandırması (.env)	7
3 • Görsel Standardizasyonu (Style Bible)	8
3.1 Boyut, Format, Çözünürlük	8
3.2 Işık, Açı, Kompozisyon Kuralları	9
3.3 AI Üretim Prompt Şablonu	10
3.4 Kalite Kontrol Kriterleri	11
4 • Pipeline Script Kodu	12
4.1 Yapılandırma ve Bağımlılıklar	12
4.2 Kaynak İstemcileri (API Clients)	13
4.3 AI Üretim Modülü	15
4.4 Standardizasyon ve İşleme	16
4.5 Ana Pipeline Döngüsü	17
5 • Çalıştırma ve Operasyon	18

Bu Doküman Kimin İçin?

Bu kılavuz, Pratik Tarifler uygulamasının 2500 tariflik görsel içeriğini üretecek geliştiriciler içindir. Hangi API'lerin nereden alınacağı, görsellerin hangi standartta olması gerektiği ve tüm süreci otomatikleştiren **çalışan script kodu** burada açıklanır.

BÖLÜM 1

Genel Bakış ve Strateji

2500 tarifin her birine, **aynı estetik standartta** profesyonel kalitede bir görsel gerekir. Bunları manuel hazırlamak haftalar sürer ve tutarsız sonuç verir. Çözüm: **hibrit kaynak stratejisi** ile yarı otomatik üretim.

1.1 Hibrit Kaynak Yaklaşımı

Pipeline, her tarif için kaynakları **öncelik sırasıyla** dener. Ücretsiz stok fotoğraf siteleri önce taranır; yeterli kalitede sonuç bulunamazsa yapay zeka ile görsel üretilir. Bu yaklaşım maliyeti minimumda tutar.

- Unsplash** — en kaliteli, telifsiz; yaygın yemeklerin ~%45'i bulunur
- Pexels** — Unsplash bulamazsa devreye girer
- Pixabay** — geniş havuz, kalite değişken; son ücretsiz şans
- DALL·E 3 (AI)** — yöresel Türk yemekleri için özel prompt'la üretim
- Flux Pro (AI)** — alternatif, daha fotogerçekçi AI motoru
- Manuel** — admin onayı / özel çekim (son çare)

💡 Neden Hibrit?

"Pizza", "salata" gibi yaygın yemekler stok sitelerinde bol bulunur — bunlar ücretsizdir. Ama "perde pilavı", "su böreği" gibi yöresel yemekler nadiren bulunur; bunlar için DALL·E 3, "geleneksel Türk sunumu" yönergesiyle özgün görsel üretir.

1.2 Maliyet ve Dağılım Tahmini

Kaynak	Pay	Tahmini Adet	Birim	Toplam
Unsplash	%45	1.125	\$0	\$0
Pexels	%15	375	\$0	\$0
Pixabay	%5	125	\$0	\$0
DALL·E 3 (standart)	%25	625	\$0,040	~\$25
Flux Pro	%10	250	\$0,055	~\$14
TOPLAM	%100	2.500	—	~\$40–60

Not: DALL·E 3 standart kalite görsel başına \$0,040; HD kalite \$0,080. Flux Pro \$0,055. Tüm 2500 tarif için toplam görsel maliyeti **\$40–60** aralığındadır — ücretsiz kaynaklar maliyeti dramatik biçimde düşürür.

📦 Depolama Maliyeti

2500 görsel $\times$ ~150 KB (full) + thumbnail + blur = **~450 MB toplam**. Firebase Storage'da bu, aylık birkaç sentlik maliyet demektir. CDN trafięi, kullanıcı sayısına göre artar.

BÖLÜM 2

Görsel Kaynak Kanalları ve API Anahtarları

Bu bölüm, her kaynağın API anahtarının **nereden ve nasıl alınacağını**, ücretlendirme ve oran limitlerini açıklar.

2.1 Unsplash API



Unsplash

ÜCRETSİZ

En yüksek kaliteli telifsiz fotoğraf kaynağı. Birincil tercih. Atıf (fotoğrafçı adı) gösterimi zorunludur.

API Anahtarı Nasıl Alınır?

- 1 unsplash.com/developers adresine gidin, hesap açın.
- 2 "New Application" → kullanım koşullarını kabul edin.
- 3 Uygulama adı ve açıklaması girin (örn. "Pratik Tarifler — recipe images").
- 4 Oluşan ekranda **Access Key**'i kopyalayın.
- 5 Başlangıçta "Demo" tier (50 istek/saat). Üretim için "Apply for Production" ile başvurun — onaylanınca limit 5.000 istek/saate çıkar.

Özellik	Değer
Endpoint	https://api.unsplash.com/search/photos
Kimlik doğrulama	<code>Authorization: Client-ID {ACCESS_KEY}</code>
Demo limiti	50 istek/saat
Üretim limiti	5.000 istek/saat
Maliyet	Ücretsiz
Env değişkeni	<code>UNSPLASH_ACCESS_KEY</code>



Unsplash Atıf Kuralı

Unsplash, indirme işleminde `download_location` endpoint'inin tetiklenmesini şart koşar. Ayrıca fotoğrafçı adı ve profili uygulamada gösterilmelidir. Pipeline kodu bunu otomatik yapar (Bölüm 4.2).

2.2 Pexels API



Pexels

ÜCRETSİZ

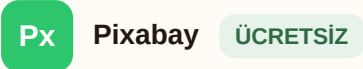
Unsplash'a güçlü bir alternatif. Atıf zorunlu değildir ama önerilir.

API Anahtarı Nasıl Alınır?

- 1 pexels.com/api adresine gidin, "Get Started"a tıklayın.
- 2 Hesap oluřturun veya giriř yapın.
- 3 Kullanım amacını kısaca yazın (örn. "mobile recipe app").
- 4 API anahtarı anında üretilir — panodan kopyalayın.

Özellik	Değer
Endpoint	https://api.pexels.com/v1/search
Kimlik doğrulama	Authorization: {API_KEY}
Limit	200 istek/saat · 20.000 istek/ay
Maliyet	Ücretsiz
Env değişkeni	PEXELS_API_KEY

2.3 Pixabay API



Geniş havuz, kalite değişken. Üçüncül ücretsiz kaynak — diğer ikisi başarısız olduğunda denenir.

API Anahtarı Nasıl Alınır?

- 1 pixabay.com/api/docs adresine gidin.
- 2 Pixabay hesabı açın ve giriş yapın.
- 3 API dokümantasyon sayfasında anahtarınız otomatik gösterilir.

Özellik	Değer
Endpoint	https://pixabay.com/api/
Kimlik doğrulama	URL parametresi: ?key={API_KEY}
Limit	100 istek/dakika
Maliyet	Ücretsiz
Env değişkeni	PIXABAY_API_KEY

2.4 OpenAI DALL·E 3 — AI Görsel Üretimi

AI

OpenAI DALL·E 3

ÜCRETLİ

Ücretsiz kaynaklarda bulunamayan yöresel yemekler için. Özel prompt mühendisliğiyle tutarlı, özgün görseller üretir.

API Anahtarı Nasıl Alınır?

- 1 `platform.openai.com` adresine gidin, hesap açın.
- 2 "Billing" bölümünden ödeme yöntemi ekleyin ve kredi yükleyin (min. \$5).
- 3 "API Keys" → "Create new secret key" ile anahtar üretin.
- 4 Anahtarı **hemen kopyalayın** — bir daha gösterilmez.
- 5 Kullanım limitlerini "Limits" bölümünden kontrol edin (Tier 1: 50 istek/dk).

Özellik	Değer
Endpoint	<code>https://api.openai.com/v1/images/generations</code>
Kimlik doğrulama	<code>Authorization: Bearer {API_KEY}</code>
Model	<code>dall-e-3</code>
Boyut seçenekleri	1024×1024 · 1792×1024 · 1024×1792
Maliyet (standart)	\$0,040 / görsel
Maliyet (HD)	\$0,080 / görsel
Oran limiti	Tier 1: 50 istek/dakika
Env değişkeni	<code>OPENAI_API_KEY</code>



Güvenlik Uyarısı

OpenAI anahtarı ücretli bir kaynaktır. Asla kod içine gömmeyin, asla Git'e commit'lemeyin. Yalnızca `.env` dosyasında tutun ve `.gitignore` 'a ekleyin. Sızması durumunda hemen iptal edip yenileyin.

2.5 Flux Pro — Alternatif AI Motoru

Fx

Flux Pro (Black Forest Labs)

ÜCRETLİ

DALL·E 3'e göre daha fotogerçekçi sonuçlar. Replicate veya fal.ai üzerinden erişilir. AI üretimi gereken görsellerin bir kısmında kalite karşılaştırması için kullanılır.

Erişim

- 1 `replicate.com` veya `fal.ai` hesabı açın.
- 2 Ödeme yöntemi ekleyin.
- 3 API token üretin (panodan kopyalayın).
- 4 Model: `black-forest-labs/flux-1.1-pro`

Özellik	Değer
Sağlayıcı	Replicate / fal.ai
Maliyet	~\$0,055 / görsel
Env değişkeni	FLUX_API_TOKEN

2.6 API Anahtarı Yapılandırması

Tüm anahtarlar proje kökünde bir `.env` dosyasında tutulur. Pipeline bu dosyayı `python-dotenv` ile okur.

Örnek `.env` :

```
# .env – API anahtarları (ASLA Git'e eklemeyin!)
# Ücretsiz stok fotoğraf kaynakları
UNSPLASH_ACCESS_KEY="abc123_unsplash_access_key"
PEXELS_API_KEY="563492_pexels_api_key"
PIXABAY_API_KEY="12345678_pixabay_key"

# AI üretim kaynakları
OPENAI_API_KEY="sk-proj-xxxxxxxxxxxxxxxxx"
FLUX_API_TOKEN="r8_xxxxxxxxxxxxxxxxxx"

# Firebase (görsel yükleme için)
FIREBASE_STORAGE_BUCKET="pratik-tarifler.appspot.com"
GOOGLE_APPLICATION_CREDENTIALS="./service-account.json"
```

✓ .gitignore Kontrolü

`.env` ve `service-account.json` mutlaka `.gitignore` dosyasına eklenmelidir. Sürüm kontrolüne sadece `.env.example` (anahtarsız şablon) girilir.

BÖLÜM 3

Görsel Standardizasyonu (Style Bible)

Tüm 2500 görsel — kaynağı ne olursa olsun — **aynı görsel dilde** olmalıdır. Bu bölüm, boyut/format/stil kurallarını tanımlar. Bu kurallar hem stok fotoğraf seçiminde hem AI üretiminde uygulanır.

3.1 Boyut, Format, Çözünürlük

Her tarif için **üç boyut** üretilir:

FULL — ANA GÖRSEL**1200 × 900 px**4:3 oran · WebP kalite 85 · ~150 KB ·
tarif detay ekranı**THUMB — KÜÇÜK GÖRSEL****400 × 300 px**4:3 oran · WebP kalite 80 · ~25 KB ·
liste/grid**BLUR — PLACEHOLDER****BlurHash string**~30 karakter · progressif yükleme
önizlemesi

Parametre	Değer	Gerekçe
En-boy oranı	4:3 (yatay)	Mobil kart ve detay ekranı için ideal
Format	WebP (JPG yedek)	%30 daha küçük dosya, geniş destek
Renk profili	sRGB	Tüm cihazlarda tutarlı renk
Full kalite	WebP q85	Kalite/boyut dengesi
Hedef dosya boyutu	~150 KB (full)	Hızlı yükleme, düşük veri
Toplam depolama	~450 MB	2500 × (full+thumb+blur)

3.2 Işık, Açı, Kompozisyon Kuralları

Görsel kimliğin tutarlı olması için **her görsel** şu kurallara uymalıdır:

Kural	Standart
Çekim açısı	Top-down (kuş bakışı 90°) — yemeğe göre 45° opsiyonel
Işık	Doğal, yumuşak, soldan gelen — sert gölge yok
Arka plan	Tarafsız: ahşap masa, mermer, açık gri veya beyaz
Kompozisyon	Yemek %70 · prop (çatal, peçete) %20 · boşluk %10
Renk paleti	Sıcak tonlar (Türk yemekleri) — soğuk (modern tabaklar)
Doku	Net, gerçekçi, abartısız — yemek iştah açıcı görünmeli
Sunum tarzı	Sade, gerçekçi — minimal Instagram estetiği

✓ YAPILACAKLAR

- Doğal ışık, yumuşak gölge

✗ KAÇINILACAKLAR

- Marka/logo görünen objeler (kola şişesi vb.)

- Tarafsız ahşap/mermer zemin
- Yemek kadrajın merkezinde, dolgun
- Gerçekçi porsiyon ve sunum
- Tutarlı 4:3 yatay çerçeve

- Aşırı şık restoran sunumu (gerçekçi değil)
- Çizgi/karikatür tarz
- Görsel üzerinde yazı veya grafik
- Ağır Instagram filtreleri, aşırı işleme
- AI filigranı veya stok foto damgası

3.3 AI Üretim Prompt Şablonu

DALL·E 3 / Flux ile üretilen görsellerin stok fotoğraflarla **tutarlı** olması için standart bir prompt şablonu kullanılır. Şablon, yemek adını ve mutfak bağlamını dinamik olarak alır:

```
# Prompt şablonu – her AI üretiminde kullanılır
PROMPT_TEMPLATE = (
    "Professional food photography of {dish_name}, "
    "{cuisine_context}, "
    "top-down 90 degree overhead shot, "
    "natural soft window light from the left, "
    "served on a neutral ceramic plate, "
    "rustic wooden table background, "
    "realistic textures, appetizing, warm tones, "
    "minimal styling, no text, no watermark, "
    "shallow depth of field, 4:3 aspect ratio, "
    "high resolution food magazine quality"
)

# Yöresel Türk yemekleri için ek bağlam
TURKISH_CONTEXT = "traditional Turkish cuisine presentation, " \
    "authentic home-style serving"

# Örnek kullanım
prompt = PROMPT_TEMPLATE.format(
    dish_name="Su Boregi (Turkish layered pastry)",
    cuisine_context=TURKISH_CONTEXT
)
```

Ⓢ Negatif Yönergeler

Prompt'a daima "no text, no watermark, realistic, not cartoon" gibi negatif yönergeler eklenir. Bu, AI'ın çizgi-film tarzı veya üzerinde yazı olan görsel üretmesini engeller – Style Bible kuralı 3.2 ile uyumludur.

3.4 Kalite Kontrol Kriterleri

Bir görsel – stok ya da AI – pipeline tarafından kabul edilmeden önce hesaplanır. Skor eşiği **0.65**'tir; altında kalan görseller bir sonraki kaynağa devredilir.

Kriter	Kontrol	Ağırlık
Çözünürlük	Genişlik ≥ 1200 px olmalı	Zorunlu (geçemezse elenir)
En-boy oranı	4:3'e yakınlık (yatay)	%25
Popülerlik	Unsplash beğeni ≥ 30 / Pixabay indirme ≥ 100	%40
Alaka	Arama terimi eşleşme gücü	%25
İçerik filtresi	Uygunsuz içerik taraması (API tarafı)	Zorunlu
Genişlik bonusu	1600 px+ ek puan	%10

İnsan İncelemesi

AI ile üretilen tüm görseller `requires_review: true` işaretiyle bir inceleme kuyruğuna girer. Bir kişi, görseli onaylar veya yeniden üretim ister.

Stok fotoğraflar skor ≥ 0.80 ise otomatik onaylanır.

BÖLÜM 4

Pipeline Script Kodu

Bu bölüm, görsel üretim pipeline'ının **çalışan Python kodunu** içerir. Kod modüler yapıdadır: her kaynak ayrı bir istemci sınıfıdır, standardizasyon ayrı bir modüldür, ana döngü hepsini birleştirir.

4.1 Yapılandırma ve Bağımlılıklar

Önce gerekli paketler kurulur:

```
# Terminal — bağımlılık kurulumu
pip install aiohttp Pillow blurhash-python openai python-dotenv
```

config.py

```
import os
from dotenv import load_dotenv

load_dotenv() # .env dosyasını oku

class Config:
    # --- Görsel boyut standartları (Style Bible 3.1) ---
    FULL_WIDTH = 1200
    FULL_HEIGHT = 900
    THUMB_WIDTH = 400
    THUMB_HEIGHT = 300
    WEBP_QUALITY_FULL = 85
    WEBP_QUALITY_THUMB = 80
    BLURHASH_X = 4
    BLURHASH_Y = 3

    # --- Kalite eşikleri (3.4) ---
    MIN_UNSPLASH_LIKES = 30
    MIN_PIXABAY_DOWNLOADS = 100
    MATCH_SCORE_THRESHOLD = 0.65

    # --- Maliyet kontrolü ---
    DALLE_COST_PER_IMAGE = 0.040
    DALLE_MAX_PER_RUN = 100 # tek çalıştırmada max AI çağrısı

    # --- API anahtarları (.env'den) ---
    UNSPLASH_KEY = os.getenv("UNSPLASH_ACCESS_KEY")
    PEXELS_KEY = os.getenv("PEXELS_API_KEY")
    PIXABAY_KEY = os.getenv("PIXABAY_API_KEY")
    OPENAI_KEY = os.getenv("OPENAI_API_KEY")

    OUTPUT_DIR = "./output"
    API_TIMEOUT = 15
    DOWNLOAD_TIMEOUT = 30
```

4.2 Kaynak İstemcileri — Stok Fotoğraf

Her ücretsiz kaynak için ortak arayüzlü bir istemci sınıfı. Hepsi `search()` metodu sunar ve standart bir sonuç sözlüğü döndürür.

clients/unsplash_client.py

```

import aiohttp
from config import Config

class UnsplashClient:
    BASE = "https://api.unsplash.com"

    def __init__(self, key):
        self.headers = {
            "Authorization": f"Client-ID {key}",
            "Accept-Version": "v1",
        }

    async def search(self, session, query):
        """Unsplash'ta ara, en kaliteli sonucu döndür."""
        params = {
            "query": query, "per_page": 10,
            "orientation": "landscape",
            "content_filter": "high", # güvenli içerik
        }
        async with session.get(f"{self.BASE}/search/photos",
                               params=params, headers=self.headers) as resp:
            if resp.status != 200:
                return None
            data = await resp.json()

            # Skorla: beğeni sayısı + min genişlik 1200
            scored = []
            for r in data.get("results", []):
                if r["width"] < 1200:
                    continue
                score = min(r.get("likes", 0) / 200, 1.0)
                scored.append((score, r))
            if not scored:
                return None
            scored.sort(key=lambda x: -x[0])
            score, best = scored[0]
            if best.get("likes", 0) < Config.MIN_UNSPASH_LIKES:
                return None

            # ZORUNLU: Unsplash atıf kuralı – download tetikle
            try:
                async with session.get(best["links"]["download_location"],
                                       headers=self.headers):
                    pass
            except Exception:
                pass

            return {
                "url": best["urls"]["full"],
                "source": "unsplash",
                "source_id": best["id"],
                "photographer": best["user"]["name"],
                "photographer_url": best["user"]["links"]["html"],
                "license": "unsplash", "score": score,
            }

```

clients/pexels_client.py

```

import aiohttp
from config import Config

class PexelsClient:
    BASE = "https://api.pexels.com/v1"

    def __init__(self, key):
        self.headers = {"Authorization": key}

    async def search(self, session, query):
        params = {"query": query, "per_page": 10,
                  "orientation": "landscape", "size": "large"}
        async with session.get(f"{self.BASE}/search",
                               params=params, headers=self.headers) as resp:
            if resp.status != 200:
                return None
            data = await resp.json()
            photos = data.get("photos", [])
            if not photos:
                return None
            # En geniş foto = en kaliteli varsayımı
            best = max(photos, key=lambda p: p["width"])
            if best["width"] < 1200:
                return None
            return {
                "url": best["src"]["original"],
                "source": "pexels", "source_id": str(best["id"]),
                "photographer": best["photographer"],
                "photographer_url": best["photographer_url"],
                "license": "pexels", "score": 0.75,
            }

```

clients/pixabay_client.py

```

import aiohttp
from config import Config

class PixabayClient:
    BASE = "https://pixabay.com/api/"

    def __init__(self, key):
        self.key = key

    async def search(self, session, query):
        params = {
            "key": self.key, "q": query,
            "image_type": "photo", "orientation": "horizontal",
            "category": "food", "per_page": 10,
            "safesearch": "true", "min_width": 1200,
        }
        async with session.get(self.BASE, params=params) as resp:
            if resp.status != 200:
                return None
            data = await resp.json()
            hits = data.get("hits", [])
            # İndirme sayısı eşiği – kalite göstergesi
            good = [h for h in hits
                    if h.get("downloads", 0) >= Config.MIN_PIXABAY_DOWNLOADS]
            if not good:
                return None
            best = max(good, key=lambda h: h["downloads"])

```

```
return {  
    "url": best["largeImageURL"],  
    "source": "pixabay", "source_id": str(best["id"]),  
    "photographer": best.get("user", ""),  
    "license": "pixabay", "score": 0.68,  
}
```

🔌 Ortak Arayüz

Üç istemci de aynı imzaya sahiptir: `async search(session, query) → dict | None`. Bu sayede ana döngü hepsini aynı şekilde, zincir halinde çağırabilir.

4.3 AI Üretim Modülü — DALL·E 3

Stok kaynaklar başarısız olduğunda devreye girer. Style Bible'a uygun prompt şablonuyla görsel üretir.

clients/ai_generator.py

```
from openai import AsyncOpenAI
from config import Config

# Style Bible uyumlu prompt şablonu (Bölüm 3.3)
PROMPT_TEMPLATE = (
    "Professional food photography of {dish}, "
    "{context}, top-down 90 degree overhead shot, "
    "natural soft window light from the left, "
    "served on a neutral ceramic plate, "
    "rustic wooden table background, realistic textures, "
    "appetizing, warm tones, minimal styling, "
    "no text, no watermark, not cartoon, "
    "shallow depth of field, high resolution, "
    "food magazine quality"
)

TURKISH_CTX = ("traditional Turkish cuisine presentation, "
               "authentic home-style serving")
GENERIC_CTX = "clean modern food styling"

class AIGenerator:
    def __init__(self, api_key):
        self.client = AsyncOpenAI(api_key=api_key)
        self.calls_made = 0

    async def generate(self, dish_name, cuisine):
        """Tarif için AI görseli üret. Maliyet kontrollü."""
        # Maliyet koruması – run başına limit
        if self.calls_made >= Config.DALLE_MAX_PER_RUN:
            raise RuntimeError("AI çağrı limiti doldu")

        ctx = TURKISH_CTX if cuisine == "turkish" else GENERIC_CTX
        prompt = PROMPT_TEMPLATE.format(dish=dish_name, context=ctx)

        resp = await self.client.images.generate(
            model="dall-e-3",
            prompt=prompt,
            size="1792x1024", # 4:3'e en yakın yatay
            quality="standard", # $0.040 (hd: $0.080)
            n=1,
        )
        self.calls_made += 1
        return {
            "url": resp.data[0].url,
            "source": "dalle",
            "source_id": f"dalle-{dish_name[:20]}",
            "license": "ai-generated",
            "score": 0.80,
            "requires_review": True, # AI görseli → insan onayı
        }
```

Maliyet Koruması

DALLE_MAX_PER_RUN sınırı, tek bir hatalı çalıştırmanın yüzlerce dolarlık fatura çıkarmasını engeller. Her AI çağrısı sayılır; limit dolunca pipeline durur ve kalan işler bir sonraki çalıştırmaya bırakılır.

4.4 Standardizasyon ve İşleme

Kaynaktan gelen ham görsel, Style Bible'a uygun hale getirilir: 1200×900 kırpma, WebP'ye çevirme, thumbnail ve BlurHash üretimi.

processing/standardize.py

```
from io import BytesIO
from PIL import Image
import blurhash
from config import Config

def standardize(raw_bytes, recipe_id, out_dir):
    """Ham görseli 3 standart çıktıya dönüştür."""
    img = Image.open(BytesIO(raw_bytes)).convert("RGB")

    # 1) Merkeze göre 4:3 kırp
    img = crop_to_ratio(img, 4, 3)

    # 2) FULL – 1200x900 WebP q85
    full = img.resize((Config.FULL_WIDTH, Config.FULL_HEIGHT),
                      Image.LANCZOS)
    full.save(f"{out_dir}/{recipe_id}_full.webp",
              "WEBP", quality=Config.WEBP_QUALITY_FULL)

    # 3) THUMB – 400x300 WebP q80
    thumb = img.resize((Config.THUMB_WIDTH, Config.THUMB_HEIGHT),
                       Image.LANCZOS)
    thumb.save(f"{out_dir}/{recipe_id}_thumb.webp",
               "WEBP", quality=Config.WEBP_QUALITY_THUMB)

    # 4) BLURHASH – progressif yükleme placeholder'ı
    small = img.resize((32, 24))
    bhash = blurhash.encode(small, Config.BLURHASH_X,
                             Config.BLURHASH_Y)

    return {
        "width": Config.FULL_WIDTH,
        "height": Config.FULL_HEIGHT,
        "blur_hash": bhash,
    }

def crop_to_ratio(img, rw, rh):
    """Görseli merkeze göre hedef orana kırp."""
    w, h = img.size
    target = rw / rh
    if w / h > target:
        # çok geniş → yanları kes
        new_w = int(h * target)
        x = (w - new_w) // 2
        return img.crop((x, 0, x + new_w, h))
    else:
        # çok uzun → alt/üst kes
        new_h = int(w / target)
        y = (h - new_h) // 2
        return img.crop((0, y, w, y + new_h))
```

4.5 Ana Pipeline Döngüsü

Tüm parçaları birleştiren ana script. Kaynakları öncelik sırasıyla dener, standardize eder, sonucu kaydeder.

pipeline.py

```
import asyncio, aiohttp, json
from config import Config
from clients.unsplash_client import UnsplashClient
from clients.pexels_client import PexelsClient
from clients.pixabay_client import PixabayClient
from clients.ai_generator import AIGenerator
from processing.standardize import standardize

async def process_recipe(session, recipe, clients, ai_gen):
    """Tek tarif için görsel üret - kaynak zinciri."""
    query = recipe["search_query"] # örn "lentil soup turkish"
    result = None

    # 1-3: Ücretsiz kaynakları sırayla dene
    for client in clients: # [unsplash, pexels, pixabay]
        result = await client.search(session, query)
        if result and result["score"] >= Config.MATCH_SCORE_THRESHOLD:
            break
        result = None

    # 4: Hiçbiri yetmezse AI üret
    if result is None:
        result = await ai_gen.generate(
            recipe["title_en"], recipe["cuisine"])

    # Görseli indir
    async with session.get(result["url"]) as resp:
        raw = await resp.read()

    # Standardize et (1200x900 WebP + thumb + blur)
    meta = standardize(raw, recipe["id"], Config.OUTPUT_DIR)
    result.update(meta)
    result["recipe_id"] = recipe["id"]
    result["status"] = "ready"
    return result

async def main(recipes):
    """Tüm tarifleri paralel işle (5'erli grup)."""
    clients = [
        UnsplashClient(Config.UNSPLASH_KEY),
        PexelsClient(Config.PEXELS_KEY),
        PixabayClient(Config.PIXABAY_KEY),
    ]
    ai_gen = AIGenerator(Config.OPENAI_KEY)
    results = []

    async with aiohttp.ClientSession() as session:
        # 5 eşzamanlı - API limitlerini aşmadan
        sem = asyncio.Semaphore(5)
        async def guarded(r):
            async with sem:
                try:
                    return await process_recipe(
                        session, r, clients, ai_gen)
                except Exception as e:
                    return {"recipe_id": r["id"],
                        "status": "failed", "error": str(e)}
        results = await asyncio.gather(
            *[guarded(r) for r in recipes])
```

```
# Sonuçları kaydet
with open("results.json", "w") as f:
    json.dump(results, f, ensure_ascii=False, indent=2)
ok = sum(1 for r in results if r["status"] == "ready")
print(f"Tamamlandı: {ok}/{len(results)} başarılı")

if __name__ == "__main__":
    with open("recipes.json") as f:
        recipes = json.load(f)
    asyncio.run(main(recipes))
```

⚡ Paralel İşleme

Semaphore(5) ile aynı anda en fazla 5 tarif işlenir. Bu, API oran limitlerini aşmadan ~2500 görseli birkaç saatte tamamlar — tek iş parçacıklı yaklaşıma göre ~5 kat hızlı.

BÖLÜM 5

Çalıştırma ve Operasyon

5.1 Adım Adım Çalıştırma

```
# 1. Bağımlılıkları kur
pip install aiohttp Pillow blurhash-python openai python-dotenv

# 2. .env dosyasını hazırla (API anahtarları)
cp .env.example .env
# .env dosyasını düzenle, anahtarları gir

# 3. Önce küçük pilot grupta test et (50 tarif)
python pipeline.py --input pilot_batch.json

# 4. Sonuçları incele – output/ klasörü + results.json

# 5. Sorun yoksa tam çalıştır (2500 tarif)
python pipeline.py --input recipes_all.json

# 6. AI görsellerini onayla (inceleme kuyruğu)
python review_ai_images.py

# 7. Onaylanan görselleri Firebase'e yükle
python upload_to_firebase.py
```

5.2 İş Kuyruğu ve Devam Edilebilirlik

Her tarif için bir **iş kaydı** tutulur (`image_jobs.json`). Durum alanları: `pending` , `processing` , `completed` , `failed` . Pipeline kesilse bile `pending` işlerden devam eder — tamamlanmış işler tekrar işlenmez.

5.3 Önerilen Çalıştırma Sırası

Aşama	Kapsam	Amaç
1. Dry-run	API çağrısı yok	Yapılandırma testi
2. Pilot	50 tarif	Kalite ve maliyet ölçümü
3. Ücretsiz tur	2500 tarif, AI kapalı	Stok kaynakları topla
4. AI turu	Kalan tarifler	Eksikleri AI ile tamamla
5. İnceleme	AI görselleri	İnsan onayı
6. Yükleme	Tüm onaylılar	Firestore Storage + Firestore

5.4 İzleme ve Maliyet Takibi

- results.json** — her tarifin kaynağı, skoru, durumu
- AI çağrı sayacı** — `AIGenerator.calls_made` ile anlık maliyet
- Başarısızlar** — `status: failed` olanlar ayrı dosyaya, tekrar denenir

- **OpenAI panosu** — platform.openai.com/usage'dan gerçek harcama

Özet

Bu pipeline, 2500 tariflik görsel içeriği **\$40–60 maliyetle** ve birkaç saatte üretir. Hibrit kaynak stratejisi (ücretsiz stok + AI) maliyeti düşük tutar; Style Bible kuralları tüm görsellerin tutarlı olmasını sağlar; iş kuyruğu sistemi süreci kesintiye dayanıklı kılar.

Geliştiriciye düşen: API anahtarlarını edinmek, `.env`'i doldurmak, pilot grupla test etmek ve tam çalıştırmadır. AI görselleri için kısa bir insan inceleme adımı önerilir.